

Yoda Otonom Ve Yazılım

Data Types (Veri Tipleri):

- Text Type:** Str: aslında metindir diyebiliriz.ve metinler her zaman tırnak içinde yazılır.
ad = "Ahmet"
yas = "25" (tırnak içinde sayı da olsa stringdir.)
- Numeric Types:** İnt: integer dan gelir aslında tam sayılarda yani ondalık kısmı olmayan sayılar için kullanılır. (int(x) # 25 (int))
Float: Floatda aslında ondalık kısmı olan ve tam olmayan sayılar için kullanılır deriz. (Float(x) # 25.0 (float))
Complex: aslında Türkçeye de geçmiş olan kompleks, karmaşık anlamında kullanırız. (a+bj)
- Sequence Types:** List: Aslında adı üstünde elimizde bulunan kelimeleri veya sayıları liste haline getirmek demektir. (x = ["apple", "banana", "cherry"])
Tuple:Aslında tuple da bir listeleme çeşididir. Fakat List ile arasındaki fark Tuple'a yazılan veriler değiştirilemez iken Liste yazılan veriler değiştirilebilir. Bu ne demektir dersek listde çıkarma ekleme yapılabilir ama tuple da yapılamaz. Diğer farkları ise list köşeli parantaz ile yazılır iken tuple normal parantez ile yazılır.
Range: genellikle döngülerde kullanılır. (For) sıra dizisi diyebiliriz. Fakat başı dahil iken sonu dahil değildir.
Örn: for i in range(0,5):
 print(i)
0
1
2
3
4
Range (start, stop, step) bu da şu demektir. (0,10,2)
0,2,4,6,8(alt alta yazılır normalde)

Mapping Type: Dict :Aslında dictionary den gelir ve şu demektir. bir anahtarım kelimem var ve bununda değeri anlamı var demektir.

```
sozluk = {  
    "isim": "Ahmet",  
    "yas": 25,  
    "sehir": "İstanbul"
```

Not: benzersiz olmalı.mesela okul sıra ayakkabı bunlar olmaz.

Set Types: Set: Aslında list ve tuple da yaptığımız karşılaştırma burada da vardır. Peki set nedir önceden de bildiğimiz gibi küme denebilir.Set de ekleme çıkarma yapılabilir.

```
a = {1, 2, 3}
```

```
b = {3, 4, 5}
```

```
print(a | b) # Birleşim → {1, 2, 3, 4, 5}
```

```
print(a & b) # Kesişim → {3}
```

```
print(a - b) # Fark → {1, 2}
```

```
print(a ^ b) # Simetrik fark → {1, 2, 4, 5} frozenset
```

Frozenset: frozensetde kümedir fakat elemanları değiştirilemez veya kaldırılamaz.

add(), remove() gibi metodlar çalışmaz, çünkü frozenset sabittir.

Ama birleşim, kesişim gibi işlemler her ikisinde de yapılabilir.

Conditionals (if-elif-else):

- Equals: `a == b`
- Not Equals: `a != b`
- Less than: `a < b`
- Less than or equal to: `a <= b`
- Greater than: `a > b`
Greater than or equal to: `a >= b`

İf yapısı aslında şu demektir böyle olsaydı neler olurdu ve biz bundan ne çıkarırdık. Eğer bizim if li yapımız bir değer şu sayıdan büyük derse bizim şartımız artık budur.ve if li yapılarda İndentation vardır peki bu nedir eğer sen kodların başladığı kısımlara farklı space or boşluk sayısı koyarsan sana şöyle bir hata verir.(Indentation Error: unexpected indent).Örn:3 tane satır yazdık ikisinde 4 space var 1 inde yok o zmn bu şu demektir. Sonuncusu if li yapıya dahil değildir.

Elif yapısı aslında senin bulman gerekeni bulamadığında sana şans tanımasıdır diyebiliriz ne demek istedik

```
a = 33
b = 33
if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
```

Elif sana bu değeri yazmanı da sundurur. Örn: senin değerin 75 olsun ve ilk yazdığın if li satır bunu karşılamıyorsa artık Elifli yazmak zorundasın. Eğer birden fazla denklem varsa elif li yazılır çünkü Python doğru yeri bulduktan sonra devamına bakmaz.

Else, bir bakımdan nothing diyebiliriz ne demek istedim. Ben önce if li yapıyı yazdım ama o aralığa girmedi sonra elifli yapı yazdım o aralığa da girmedi bir daha yazmamda sıkıntı yok elifi ama sonunda hiçbiri gelmezse şunu yazıyoruz.

Else (Yanı boş olur oraya denklem yazılmaz.)

Print (doğru olan denklemi yazıyı buraya yazıyoruz.) Aslında mantık if ve elif bulamadıysa en son else'e doğruyu yazar ve denklem biter.Peki elif olmak zorunda mı ? Hayır çünkü if li yapıyı yazdıktan sonra direk elseli yapıyı yazabilirim. **Note:** The else statement must come last.

Loops (for, while):

i = 1 i'nin başlangıç değeri 1

while i < 6: i, 6'dan küçük olduğu sürece döngü devam eder

```
print(i) i'yi ekrana yazdır
```

i += 1 i'nin değeri her turda 1 artar.

Aslında böyle bir yazılım yaptığımızda while burdaki işlevi -ken kan anlamı katmasıdır yani 6 dan küçük olan tam sayıları yaz demektir.

Ve while yazımı kullanılırken şunlarda yapılabilir 'Break, continue işlemleri de yapılabilir ve artık while daki işlemiz bitmiş ise mesela örnekteki gibi else: yazıp yazılımı bitirebiliriz.

For: For için şunu diyebiliriz. bir listedeki veyahut bir dizideki her bir eleman için işlem yapmasını isteriz. Ve çok kullanılır.

```
for i in range(5):
```

```
print(i)
```

önceden de öğrendiğimiz gibi bu ne demektir range(5) için i yaz demektir.

For değişken in koleksiyon bu kalıbı kullanırız. For için: for harf in 'python' dediğinde print(harf) yazdıktan sonra bize bu değerleri verir.(p,y,t,h,o,n)

Functions (Fonksiyonlar):

Pythonda fonksiyon tanımlamak için 'def' anahtar kelimesi kullanılır.

def fonksiyon_adi(): bu kalıp kullanılır.Örn: def selamla(İsim):

```
print ("Merhaba! Hoş geldin.")
```

ve fonksiyonu çağırmak için selamla() yazarsın.

Aslında amacımız kodun düzenli olması, her parçanın tek işi yapması, aynı kodu birçok yerde tekrar kullanabilmen.

```
def topla (a, b):
```

```
print("toplama sonucu:", a + b)
```

```
topla(3,7)
```

```
topla(7,8)
```

```
def çarp (c, d):  
    print("Çarpma sonucu:" , c * d)
```

```
çarp(3,7)
```

```
çarp(7,8)
```

Not:İkisini de yazarken . ve , kullanımında eksiklik yaptım. Ama anladığımı düşünüyorum.

Fonksiyon yazılırken _ alt tire kullanılır çünkü kullanmazsak Python bunu iki farklı şey anlar.

```
def fahrenheit_to_celsius(f):  
    return (f - 32) * 5 / 9
```

```
print(fahrenheit_to_celsius(100))
```

```
print(fahrenheit_to_celsius(212))
```

```
print(fahrenheit_to_celsius(50))
```

return kullanım ise geri döndürmek anlamına gelir ama durdurmak demektir.

```
def kare(x):  
    return x * x
```

```
print(kare(4)) # 16
```

```
alan = kare(5)
```

```
print(alan) # 25
```

```
class Araba:
```

```
    def __init__(self, marka, model):  
        self.marka = marka  
        self.model = model
```

```
araba1 = Araba("Toyota", "Corolla")
```

```
araba2 = Araba("BMW", "X5")
```

```
print(araba1.marka)
```

```
print(araba2.model)
```

Çıktı:

Toyota

X5

Self : o nesnenin kendisini gösterir.

İnit: başlangıcı yapar diyebiliriz.